UNIVERSITÉ PARIS 1
PANTHÉON SORBONNE

Detection of Structural Breaks in Variance Using the ICSS Algorithm

An application to U.S. stock returns

Presented by:

Aya WIFAK, Alicia DAHMANI, Aya EL FARFAR

Under the supervision of:

Philippe DE PERETTI

Academic Year 2025 – 2026

Contents

1 Introduction **2**

2 Literature Review **3**

2.1 Structural breaks in variance . . . 3

2.2 Early approaches to change point detection . . . 3

2.3 The ICSS algorithm . . . 3

2.4 Extensions and applications . . . 4

3 Methodology **4**

3.1 Setup and notation . . . 4

3.2 The cumulative sum of squares and the D_k statistic . . . 4

3.3 Test statistic and decision rule . . . 5

3.4 Efficiency Factors and Limitations . . . 5

3.5 The ICSS algorithm . . . 5

3.6 Implementation . . . 7

3.7 Validation on simulated data . . . 7

4 Data **8**

4.1 Data source . . . 8

4.2 Returns computation . . . 8

4.3 Preliminary analysis . . . 8

5 Results **8**

5.1 ICSS output — USD/RUB . . . 8

5.2 Graphical analysis . . . 9

5.3 Economic interpretation . . . 10

6 Conclusion **11**

A SAS Code **13**

A.1 ICSS algorithm validation on simulated data . . . 13

A.2 Empirical Application on Russian exchange rate . . . 16

1 Introduction

The variance of financial asset returns is not constant over time. It shifts abruptly in response to crises, policy changes, or corporate events. Ignoring these structural breaks leads to biased estimates in volatility models: Lamoureux and Lastrapes (1990) showed that undetected variance shifts cause GARCH models to overestimate the persistence of volatility.

Inclán and Tiao (1994) proposed the Iterative Cumulative Sum of Squares (ICSS) algorithm to detect multiple change points in the unconditional variance of a time series. The method uses a centered cumulative sum of squares statistic and an iterative search procedure combining forward and backward passes. Its computational simplicity has made it one of the most widely used tools for structural break detection in financial econometrics.

The ICSS algorithm is implemented from scratch in SAS/IML and applied to the USD/RUB monthly exchange rate between 1995 and 2026. This period is characterized by a number of macroeconomic and geopolitical shocks, such as the invasion of Ukraine in 2022, the annexation of Crimea in 2014, and the sovereign default in 1998. To ensure the validity of the iid assumption underpinning the ICSS method, an ARMA(1,1) filter is used before the algorithm is used to handle the considerable autocorrelation present in the returns series.

The remainder of this report is structured as follows: Section 2 reviews the related literature, Section 3 presents the ICSS methodology and its validation on simulated data, Section 4 describes the dataset, Section 5 reports the empirical findings, and Section 6 concludes.

2 Literature Review

2.1 Structural breaks in variance

Financial time series frequently violate the assumption of constant variance. Asset returns exhibit abrupt shifts in volatility triggered by economic crises, policy changes, or institutional events. Lamoureux and Lastrapes (1990) showed that ignoring such breaks leads to an overestimation of volatility persistence in GARCH models, while Diebold (1986) noted that apparent long memory in volatility may be an artifact of undetected structural breaks.

2.2 Early approaches to change point detection

Hsu, Miller, and Wichern (1974) were among the first to model stock returns using a normal probability model with nonstationary variance, as an alternative to heavy-tailed distributions such as the Pareto distribution. Early methods by Hinkley (1971), Smith (1975), and Hsu (1977) focused on detecting a single change point, limited by the computational burden of searching for multiple breaks simultaneously. Wichern, Miller, and Hsu (1976) and Abraham and Wei (1984) extended this framework to autoregressive models. Baufays and Rasson (1985) introduced maximum likelihood methods for multiple breaks with reduced computational cost, but still required specifying the number of breaks in advance.

2.3 The ICSS algorithm

The ICSS algorithm was presented by Inclán and Tiao (1994) as a computationally effective method for identifying multiple variance breaks. The method's basis is the centered cumulative sum of squares D_k, which contrasts the expected proportion under constant variance with the actual proportion of variance gathered at observation k. D_k deviates from zero when a break occurs; a trough is created by an increase in variance, while a peak is created by a reduction. The test statistic $M_T = \max_k |D_k| \times \sqrt{T/2}$ follows the asymptotic distribution of a Brownian bridge, with a critical value of 1.358 at the 5% level. Instead of checking the complete series at once, the algorithm searches repeatedly on progressively smaller parts to get over the masking effect, which occurs when one powerful break hides another.

2.4 Extensions and applications

Sansó et al. (2004) proposed a modified ICSS that corrects for non-normality using a nonparametric estimate of the fourth moment, addressing the tendency of the original algorithm to detect spurious breaks under excess kurtosis. The ICSS has been widely applied in empirical finance: Aggarwal et al. (1999) on emerging markets, Malik (2003) on exchange rates, and Rapach and Strauss (2008) on U.S. dollar exchange rates. The consensus is that incorporating ICSS-detected breaks into GARCH models significantly reduces estimated volatility persistence and improves risk forecasting.

3 Methodology

3.1 Setup and notation

Consider T independent observations $\{a_t\}_{t=1}^{T}$ with zero mean and variance σ_j^2 constant within each regime, shifting at unknown change points $\kappa_1, \ldots, \kappa_{N_T}$. The main objective is to estimate both N_T and their locations.

3.2 The cumulative sum of squares and the D_k statistic

The cumulative sum of squares is $C_k = \sum_{t=1}^{k} a_t^2$. The centered statistic is:

$$D_k = \frac{C_k}{C_T} - \frac{k}{T}, \quad k = 1, \ldots, T-1 \tag{1}$$

with $D_0 = D_T = 0$. Under constant variance, D_k fluctuates around zero. An increase in variance creates a negative trough in D_k; a decrease creates a positive peak. Inclán and Tiao (1994) showed that D_k is a monotone function of the F-statistic for comparing two variances. Theoretically, the authors demonstrate that the D_k statistic is directly related to the standard F-test used to compare two variances. But unlike the F-test, the ICSS does not require the breakpoint to be known in advance. It dynamically identifies the location of the change. It is also related to the log-likelihood ratio by:

$$LR \approx \max_k \left\{ \frac{T^3 D_k^2}{4k(T-k)} \right\} \tag{2}$$

3.3 Test statistic and decision rule

For a segment $[s : e]$, define:

$$k^*(s : e) = \arg\max_{k \in [s,e]} |D_k|, \qquad M(s : e) = \max_k \sqrt{\frac{e - s + 1}{2}}\, |D_k| \tag{3}$$

Under H_0, $\sqrt{T/2}\, D_k$ converges to a Brownian bridge. The critical value at the 5% level is $D^* = 1.358$. If $M(s : e) > D^*$, a break is detected at k^*.

3.4 Efficiency Factors and Limitations

Despite its effectiveness, the ICSS algorithm is susceptible to "midpoint bias" Inclán and Tiao (1994). Its detection capability is maximized when variance changes occur in the middle of the sample (around 50%). If a break occurs very early (25%) or very late (75%), the statistical signal is automatically weaker and the break becomes more difficult to identify.

In addition, the algorithm has to deal with the "mask effect," where an extremely abrupt break can obscure more subtle changes in variance elsewhere in the series. It is precisely to counter this phenomenon that the iterative splitting solution is used, allowing the algorithm to isolate each change point by dividing the series into smaller segments until no further breaks are found.

3.5 The ICSS algorithm

The algorithm is based on a calculation function which evaluates the D_k statistic to extract the maximum M and its position $k*$, and takes place in two main phases:

1. Candidate breakpoints search (Phase A)
2. Breakpoints refinement and validation (Phase B).

Algorithm 1: Iterated Cumulative Sums of Squares (ICSS) Algorithm

```
procedure CALC-D(a, t_1, t_2)
    n ← t_2 − t_1 + 1
    if n < 2 then
        return 0, t_1
```

```
    end if
    a_sub ← a[t_1 ... t_2] − mean(a[t_1 ... t_2])
    for k ← 1 ... n do
        C_k ← Σ_{i=1}^{k} a_sub[i]^2
        D_k ← C_k / C_n − k / n
    end for
    k* ← arg max_k |D_k| + t_1 − 1
    M ← √(n/2) max_k |D_k|
    return M, k*
end procedure

procedure PHASE-A(a, T, D*)
    Q ← {[1, T]}
    CP ← empty list
    repeat
        [s, e] ← pop first segment from Q
        M, k* ← CALC-D(a, s, e)
        if M > D* then
            k_first ← k*
            repeat
                M, k ← CALC-D(a, s, k_first)
                if M > D* then k_first ← k
                end if
            until M ≤ D*
            k_last ← k*
            repeat
                M, k ← CALC-D(a, k_last + 1, e)
                if M > D* then k_last ← k
                end if
            until M ≤ D*
            Add k_first to CP
            if k_first ≠ k_last then
                Add k_last to CP
                if k_last − k_first > 2 then
                    Add [k_first + 1, k_last] to Q
                end if
            end if
        end if
    until Q is empty
```

```
    return sorted CP
end procedure

procedure PHASE-B(a, T, D*, CP)
    repeat
        CP_old ← CP
        CP_new ← empty list
        CP_full ← add boundaries 0 and T to CP
        for each breakpoint cp_j in CP_full do
            s ← cp_{j-1} + 1
            e ← cp_{j+1}
            M, k ← CALC-D(a, s, e)
            if M > D* then
                Add k to CP_new
            end if
        end for
        CP ← CP_new
        max_shift ← max(|CP − CP_old|)
    until number of points is stable and max_shift ≤ 2
    return CP
end procedure
```

3.6 Implementation

The algorithm is implemented in SAS/IML using a reusable module that tests any segment $[L : R]$ and returns either the break location or zero. The forward pass, backward pass, and refinement loop call this module with different boundaries. The complete code is provided in Appendix A.

3.7 Validation on simulated data

To verify the implementation, we generate $T = 600$ observations with three variance regimes ($\sigma^2 = 1, 3, 1$) and true break points at $t = 200$ and $t = 400$. The ICSS algorithm detects two breaks at $t = 186$ and $t = 400$, correctly identifying both the number and approximate location of the breaks. The 14-observation discrepancy on the first break is within the expected precision of the method on finite samples.

Table 1: ICSS validation on simulated data

Break	True location	Detected location
1	200	186
2	400	400

4 Data

4.1 Data source

For the USD/RUB exchange rate, we use monthly exchange rates from Investing.com, which extend from January 1995 to March 2026 (377 observations). This dataset is useful for variance break identification as it contains a number of significant macroeconomic and geopolitical moments in Russian financial history.

4.2 Returns computation

Raw exchange rates are non-stationary, so we compute logarithmic returns:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) \tag{4}$$

Log-returns are approximately stationary, additive over time, and symmetric. The first observation is lost due to the lag, yielding T = 376 observations.

4.3 Preliminary analysis

Two diagnostic tests are carried out before the ICSS algorithm is applied. For all lag parameters and model types, the ADF test verifies that log-returns are stationary ($p < 0.0001$). However, the Ljung-Box test reveals significant autocorrelation at multiple lags ($p < 0.0001$), violating the iid assumption underlying the standard ICSS procedure. In order to solve this, the returns series is subjected to an ARMA(1,1) filter, and the residuals are then subjected to the ICSS method.

5 Results

5.1 ICSS output — USD/RUB

The ICSS algorithm, applied to the residuals of an ARMA(1,1) filter, detects four structural breaks at observations 8, 51, 257, and 333, corresponding to January 1995, January 1999, January 2016, and January 2022.

Table 2: Variance regimes detected by ICSS — USD/RUB

Regime	**Break**	**Obs.**	**Date**	**Rate**	**Variance**	**Std. Dev.**
1	1	8	06 Jan 1995	4.5370	0.0055	0.0745
2	2	51	01 Jan 1999	23.1000	0.0101	0.1004
3	3	257	03 Jan 2016	67.0288	0.0015	0.0387
4	4	333	07 Jan 2022	61.6200	0.0035	0.0587
5	—	—	—	—	0.0027	0.0516

5.2 Graphical analysis

The residuals plot displays heterogeneous volatility across the sample, with notably larger amplitude around observation 51 (January 1999, variance = 0.0101) and a prolonged period of low volatility between observations 51 and 257 (variance = 0.0015). A second surge is visible around observation 333 (January 2022, variance = 0.0035).

The C_k plot exhibits four distinct changes in slope at observations 8, 51, 257, and 333, with the steepest segment corresponding to Regime 2 and the flattest to Regime 3, consistent with their respective variance estimates of 0.0101 and 0.0015.

The D_k statistic confirms that the structural changes in USD/RUB volatility are statistically significant. It oscillates between about 0.08 and +0.04, with four significant direction changes that exactly match the four observed breaks.

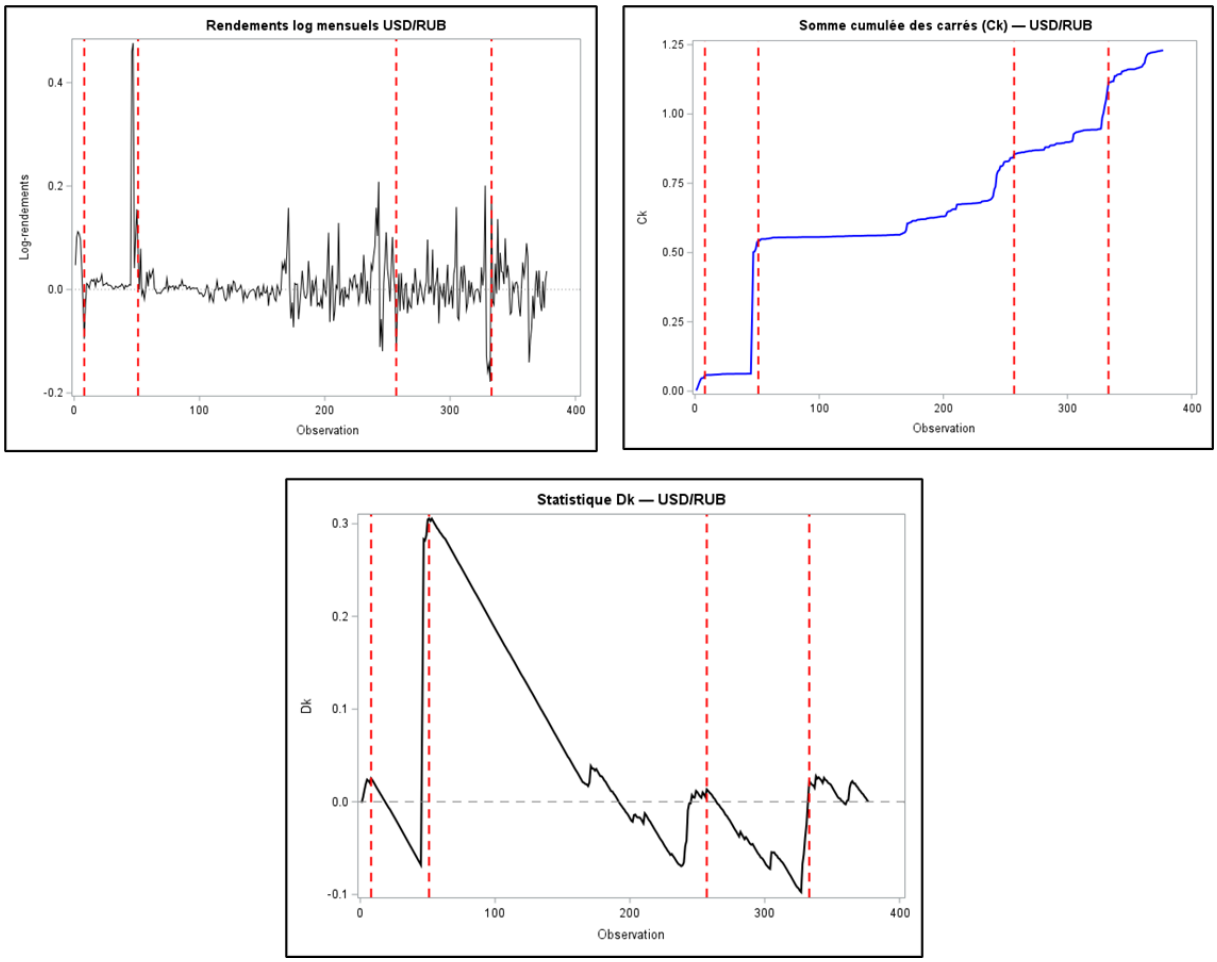

Figure 1: ICSS diagnostic plots for USD/RUB

5.3 Economic interpretation

The four observed breakdowns line up with prevalent geopolitical and macroeconomic shifts in Russian economic history. In the early post-Soviet transition period, major financial instability marked by hyperinflation and a precarious banking sector began with the first break (January 1995, observation 8). The second break (January 1999, observation 51) comes after the August 1998 Russian sovereign collapse, which led to a major increase in exchange rate volatility and a severe devaluation of the rouble, as seen by the sample's largest variance (0.0101).

The annexation of Crimea in 2014, Western sanctions, and the dramatic drop in oil prices in 2015–2016 all contributed to the third break (January 2016, observation 257), which put fresh pressure on the rouble following almost ten years of stability (Regime 3, variance = 0.0015). The invasion of Ukraine and the subsequent, unprecedented wave of international sanctions correlate to the fourth rupture (January 2022, observation 333). Regime 5 records a moderate variance (0.0027) despite the shock's intensity, which is probably due to the Russian central bank's emergency measures in early 2022, such as capital controls and strong interest rate rises.

Overall, the ICSS results confirm that USD/RUB volatility is driven by discrete exogenous shocks, highlighting the sensitivity of market exchange rates to geopolitical and macroeconomic disruptions.

6 Conclusion

This report implemented the ICSS algorithm of Inclán and Tiao (1994) in SAS/IML and applied it to the monthly USD/RUB exchange rate returns sourced from Investing.com over the period 1995–2026.

Actually, five different variance regimes occurred through the identification of four structural breaches in January 1995, January 1999, January 2016, and January 2022. Every breach coincides with a significant geopolitical or macroeconomic shock, demonstrating how vulnerable developing market currency rates are to external shocks. To ensure the validity of the iid assumption underpinning the technique, an ARMA(1,1) filter was used to address the considerable autocorrelation in the returns series before to applying the ICSS.

References

Aggarwal, R., Inclán, C., and Leal, R. (1999). Volatility in emerging stock markets. *Journal of Financial and Quantitative Analysis.*

Hinkley, D. V. (1971). Inference about the change-point from cumulative sum tests. *Biometrika.*

Inclán, C. and Tiao, G. C. (1994). Use of cumulative sums of squares for retrospective detection of changes of variance. *Journal of the American Statistical Association.*

Lamoureux, C. G. and Lastrapes, W. D. (1990). Persistence in variance, structural change, and the GARCH model. *Journal of Business & Economic Statistics.*

Malik, F. (2003). Sudden changes in variance and volatility persistence in foreign exchange markets. *Journal of Multinational Financial Management.*

Rapach, D. E. and Strauss, J. K. (2008). Structural breaks and GARCH models of exchange rate volatility. *Journal of Applied Econometrics.*

Sansó, A., Aragó, V., and Carrion-i-Silvestre, J. L. (2004). Testing for changes in the unconditional variance of financial time series.

A SAS Code

A.1 ICSS algorithm validation on simulated data

```
proc iml;

    call randseed(12345);
    n_total = 600;
    a = j(n_total, 1, 0);
    a[1:200] = randnormal(200, 0, 1);
    a[201:400] = randnormal(200, 0, 3);
    a[401:600] = randnormal(200, 0, 1);

    n_total = nrow(a);
    a2 = a ## 2;
    D_critique = 1.358;

    start icss_test(a2, L, R) global(D_critique);
        seg = a2[L:R];
        N = R - L + 1;
        C = cusum(seg);
        C_N = C[N];
        k = T(1:N);
        D = (C / C_N) - (k / N);
        absD = abs(D);
        k_star = absD[<:>];
        M_N = max(absD) * sqrt(N / 2);
        if M_N > D_critique then return(L + k_star - 1);
        else return(0);
    finish;

    cp_fwd = j(50, 1, 0);
    n_fwd = 0;
    L = 1;
    do while(L < n_total);
        tau = icss_test(a2, L, n_total);
        if tau = 0 then leave;
        stable = 0;
        do while(stable = 0);
            tau2 = icss_test(a2, L, tau);
            if tau2 = 0 | tau2 = tau then stable = 1;
            else tau = tau2;
        end;
        n_fwd = n_fwd + 1;
        cp_fwd[n_fwd] = tau;
        L = tau + 1;
    end;

    cp_bwd = j(50, 1, 0);
    n_bwd = 0;
    R = n_total;
    do while(R > 1);
        tau = icss_test(a2, 1, R);
        if tau = 0 then leave;
```

```
        stable = 0;
        do while(stable = 0);
            tau2 = icss_test(a2, tau + 1, R);
            if tau2 = 0 | tau2 = tau then stable = 1;
            else tau = tau2;
        end;
        n_bwd = n_bwd + 1;
        cp_bwd[n_bwd] = tau;
        R = tau;
    end;

    all_cp = j(100, 1, 0);
    n_all = 0;
    do i = 1 to n_fwd;
        n_all = n_all + 1;
        all_cp[n_all] = cp_fwd[i];
    end;
    do i = 1 to n_bwd;
        n_all = n_all + 1;
        all_cp[n_all] = cp_bwd[i];
    end;
    if n_all > 0 then do;
        all_cp = all_cp[1:n_all];
        call sort(all_cp, 1);
        unique = j(n_all, 1, 0);
        unique[1] = all_cp[1];
        n_unique = 1;
        do i = 2 to n_all;
            if all_cp[i] ^= all_cp[i - 1] then do;
                n_unique = n_unique + 1;
                unique[n_unique] = all_cp[i];
            end;
        end;
        all_cp = unique[1:n_unique];
        n_all = n_unique;
    end;

    converged = 0;
    do iter = 1 to 20 while(converged = 0);
        new_cp = j(50, 1, 0);
        n_new = 0;
        n_bounds = n_all + 2;
        bounds = j(n_bounds, 1, 0);
        bounds[1] = 0;
        do i = 1 to n_all;
            bounds[i + 1] = all_cp[i];
        end;
        bounds[n_bounds] = n_total;
        do i = 1 to n_bounds - 1;
            seg_L = bounds[i] + 1;
            seg_R = bounds[i + 1];
            tau = icss_test(a2, seg_L, seg_R);
            if tau > 0 then do;
                n_new = n_new + 1;
                new_cp[n_new] = tau;
```

```
            end;
        end;
        if n_new = 0 & n_all = 0 then converged = 1;
        else if n_new = n_all then do;
            new_sorted = new_cp[1:n_new];
            call sort(new_sorted, 1);
            if all(new_sorted = all_cp) then converged = 1;
            else do; all_cp = new_sorted; n_all = n_new; end;
        end;
        else if n_new > 0 then do;
            all_cp = new_cp[1:n_new];
            call sort(all_cp, 1);
            n_all = n_new;
        end;
        else n_all = 0;
    end;

    if n_all > 0 then do;
        result = all_cp[1:n_all];
        print result[label="Detected break points"];
        print n_all[label="Number of breaks"];
    end;
    else print "No break detected";

    C = cusum(a2);
    C_N = C[n_total];
    k = T(1:n_total);
    D = (C / C_N) - (k / n_total);

    graphmat = k || a || C || D;
    create graphdata from graphmat[colname={"t" "a_t" "C_k" "D_k"}];
    append from graphmat;
    close graphdata;

    if n_all > 0 then do;
        create cpdata from all_cp[colname={"cp"}];
        append from all_cp;
        close cpdata;
    end;
    else do;
        cp_vide = {.};
        create cpdata from cp_vide[colname={"cp"}];
        append from cp_vide;
        close cpdata;
    end;

quit;

proc sql noprint;
    select cp into :cplist separated by ' '
    from cpdata where cp ^= .;
    select count(cp) into :ncp trimmed
    from cpdata where cp ^= .;
quit;
```

```
proc sgplot data=graphdata;
    series x=t y=a_t / lineattrs=(color=black thickness=1);
    %if &ncp > 0 %then %do;
        refline &cplist / axis=x lineattrs=(color=red pattern=dash thickness=2)
                          label=("Ruptures");
    %end;
    xaxis label="t";
    yaxis label="a(t)";
    title "Observed series with detected breaks";
run;

proc sgplot data=graphdata;
    series x=t y=C_k / lineattrs=(color=blue thickness=2);
    %if &ncp > 0 %then %do;
        refline &cplist / axis=x lineattrs=(color=red pattern=dash thickness=2);
    %end;
    xaxis label="t";
    yaxis label="Ck";
    title "Cumulative sum of squares (Ck)";
run;

proc sgplot data=graphdata;
    series x=t y=D_k / lineattrs=(color=black thickness=2);
    refline 0 / axis=y lineattrs=(color=gray pattern=dash);
    %if &ncp > 0 %then %do;
        refline &cplist / axis=x lineattrs=(color=red pattern=dash thickness=2);
    %end;
    xaxis label="t";
    yaxis label="Dk";
    title "Dk statistic";
run;
```

A.2 Empirical Application on Russian exchange rate

The implementation follows the same algorithmic structure as described in Section 3. The only modifications concern the data input and the addition of an ARMA(1,1) pre-filtering step to address the significant autocorrelation present in the returns series.

```
proc import datafile="C:\Users\ayael\Downloads\USD_RUB Historical Data (1).csv"
    out=rub_raw dbms=csv replace;
    getnames=yes;
run;

data rub_clean;
    set rub_raw (keep=Date Price);
    price_clean = input(compress(Price, ','), best12.);
    date_fmt = input(Date, ddmmyy10.);
    format date_fmt date9.;
    keep date_fmt price_clean;
    rename date_fmt = date
           price_clean = close;
```

```
run;

proc sort data=rub_clean; by date; run;

data rendements;
    set rub_clean;
    close_lag = lag(close);
    if close_lag > 0 then
        r = log(close / close_lag);
    if _n_ > 1;
    keep date close r;
run;

proc arima data=rendements;
    identify var=r stationarity=(adf=4);
run; quit;

proc arima data=rendements;
    identify var=r nlag=24;
    estimate p=1 q=1 method=ml;
    output out=residus residual=resid;
run; quit;

/* ICSS applied on residuals instead of raw returns */
proc iml;
    use residus;
    read all var {resid} into a;
    close residus;
    /* remainder identical to simulation code */
quit;
```